

Proyecto 1: Calculadora de 4 bits

Ignacio Cruzat

Juan Vicente Celedón

Wille Saarenpaa

Arquitectura de Computadores

Profesor: Jorge Gómez Mir

8 de septiembre de 2026

Índice

1. Introducción	3
2. Arquitectura general	4
2.1. Operaciones	5
3. Diseño Lógico	5
3.1. Complemento a dos y overflow	5
3.2. Full adder de 1 bit	6
3.3. Multiplexor 2:1	7
3.4. Restas	8
3.5. Shift Left y Shift Right	8
4. Implementación de las operaciones mediante compuertas	9
5. GTKWave y Simulación	9
6. FPGA	10
7. Conclusión	11

1. Introducción

El presente informe describe el diseño e implementación de una calculadora de 4 bits. El circuito opera con números enteros en complemento a dos, en un rango de -8 a 7 . La lógica se describió en Verilog mediante compuertas, se verificó con Icarus Verilog y GTKWave, y se implementó en una FPGA Lattice iCE40 HX1K de la Nandland Go Board.

El trabajo busca recorrer el flujo completo de diseño digital. En primer lugar se construyen las tablas de verdad y los mapas de Karnaugh. A partir de esas expresiones se obtiene la lógica con primitivas and, or, not y xor. Luego se describe el circuito en Verilog, se comprueba su comportamiento en simulación y, por último, se programa el diseño en el hardware.

La calculadora dispone de dos operandos de 4 bits. El primero corresponde siempre a `op1`. El segundo se selecciona con `sel_op2`: puede ingresarse un operando externo `op2_ext` o reutilizarse el resultado almacenado. Un código de 3 bits determina la operación (reinicio, suma, resta, resta inversa, shift left o shift right). El resultado se guarda en un registro de 4 bits, el cual se actualiza en el flanco del reloj cuando la señal ejecutar se encuentra activa. Si una operación excede los 4 bits, se conservan únicamente los cuatro bits menos significativos, de acuerdo con el enunciado.

En las secciones siguientes se presenta la arquitectura general, el diseño de los bloques combinacionales, la implementación mediante compuertas, los resultados de simulación y la puesta en marcha sobre la Go Board.

2. Arquitectura general

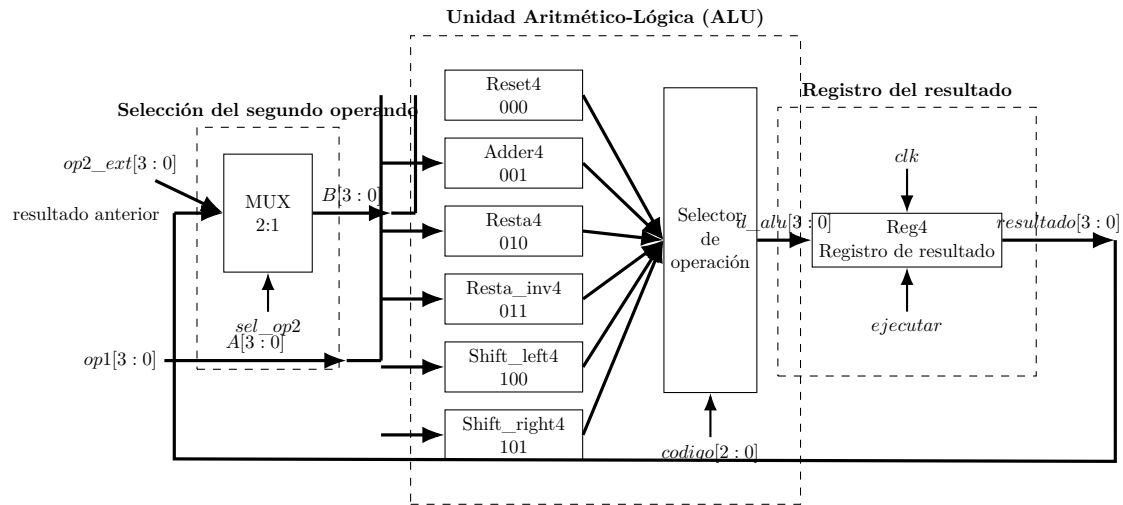


Figura 1: Arquitectura general de la calculadora de 4 bits.

Fuente: Elaboración propia.

La Figura 1 muestra la organización del circuito. El diseño se divide en tres bloques: la selección del segundo operando, la unidad aritmético-lógica y el registro de resultado.

El multiplexor 2:1 recibe `op2_ext` y el valor almacenado en el registro. La señal `sel_op2` selecciona cuál de las dos entradas ingresa a la ALU como operando *B*. El operando *A* corresponde siempre a `op1`. De este modo es posible encadenar una operación con el resultado anterior, tal como indica el enunciado.

La ALU calcula las seis operaciones en paralelo. Un decodificador de 3 bits habilita únicamente la salida asociada al código de operación. Dicha salida no se entrega de forma directa: ingresa al registro `reg4`, que actualiza su contenido en el flanco de `clk` cuando la señal `ejecutar` vale 1. Mientras `ejecutar` permanece en 0, el valor almacenado se mantiene.

El módulo de tope utilizado en el testbench y en la FPGA se denomina `calculadora_4bits`. Sus pines son `clk`, `ejecutar`, `codigo`, `sel_op2`, `op1`, `op2_ext` y `resultado`.

2.1. Operaciones

Cuadro 1: Códigos de operación implementados.

Código	Operación	Resultado
000	Reinicio	$R = 0000$
001	Suma	$R = A + B$
010	Resta	$R = A - B$
011	Resta inversa	$R = B - A$
100	Shift left	$R = A$ corrido $B[1 : 0]$ a la izquierda
101	Shift right	$R = A$ corrido $B[1 : 0]$ a la derecha

Los códigos 110 y 111 no se usan. El shift mira solo los dos bits menos significativos de B ; $B[3 : 2]$ no cambian cuánto se corre. Suma, restas y shifts quedan en 4 bits, si hay overflow, se conservan los cuatro bits menos significativos.

3. Diseño Lógico

3.1. Complemento a dos y overflow

Los operandos y el resultado se interpretan en complemento a dos de 4 bits. El bit más significativo es el signo, si vale 0 el número es positivo o cero (0 a 7), si vale 1 el número es negativo (-1 a -8).

Patron	Sin signo	Complemento a dos
0000	0	0
0001	1	1
0111	7	7
1000	8	-8
1010	10	-6
1110	14	-2
1111	15	-1

3.2. Full adder de 1 bit

El full adder calcula S (bit de suma) y C_{out} (acarreo de salida) a partir de A , B y C_{in} .

Cuadro 2: Tabla de verdad del full adder.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Expresiones booleanas:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$$

Suma S

A	BC			
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} = A \oplus B \oplus C_{in}$$

Los unos no forman grupos de dos: S se implementa con XOR.

Acarreo C_{out}

A	BC			
	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Grupos: AB , AC_{in} , BC_{in} .

$$C_{out} = AB + AC_{in} + BC_{in} = AB + C_{in}(A \oplus B)$$

3.3. Multiplexor 2:1

El multiplexor 2:1 elige el segundo operando. Si $s = 0$ sale d_0 (el op2 externo). Si $s = 1$ sale d_1 (el resultado que ya estaba guardado). En la placa, s es `sel_op2`. Se arma con un NOT, dos AND y un OR.

Se instancia cuatro veces, un mux por cada bit, para armar el B de 4 bits que entra a la ALU.

Cuadro 3: Tabla de verdad del multiplexor 2:1.

s	d_0	d_1	y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$$y = (\bar{s} \cdot d_0) + (s \cdot d_1)$$

El mapa de Karnaugh usa orden Gray en las columnas:

	d_0d_1			
s	00	01	11	10
0	0	0	1	1
1	0	1	1	0

El grupo de la fila $s = 0$ y columnas 11–10 produce $\bar{s}d_0$; el de la fila $s = 1$ y columnas 01–11 produce sd_1 . La suma lógica de ambos grupos da la expresión anterior.

3.4. Restas

$$A - B = A + \overline{B} + 1$$

De manera análoga, $B - A = B + \overline{A} + 1$. En ambos casos se invierten los bits del operando que se resta y se fija el acarreo inicial del sumador en 1. Por ello se utilizan las mismas tablas y mapas del full adder, con la entrada complementada. El acarreo final se descarta y se conservan únicamente los cuatro bits de la diferencia.

Cuadro 4: Ejemplos de resta en complemento a dos.

Cuenta	Binario	Resultado
$5 - 2$	$0101 - 0010$	0011 (3)
$2 - 5$	$0010 - 0101$	1101 (-3)
$-5 - (-2)$	$1011 - 1110$	1101 (-3)

3.5. Shift Left y Shift Right

Los desplazamientos operan sobre el operando A . La cantidad de posiciones la definen únicamente los dos bits menos significativos de B , por lo que el corrimiento puede ser de 0, 1, 2 o 3 lugares. Los bits $B[3 : 2]$ no intervienen. En ambos casos se trata de un shift lógico: los espacios que quedan libres se rellenan con ceros.

En el shift left los bits de A se desplazan hacia el bit más significativo y entran ceros por la derecha. El bit que se sale por la izquierda se pierde.

Cuadro 5: Shift left en función de $B[1 : 0]$.

$B[1 : 0]$	Resultado
00	$a_3 a_2 a_1 a_0$
01	$a_2 a_1 a_0 0$
10	$a_1 a_0 0 0$
11	$a_0 0 0 0$

En el shift right los bits se desplazan hacia el bit menos significativo y entran ceros por la izquierda. El bit que se sale por la derecha se descarta. No se copia el bit de signo.

Cuadro 6: Shift right en función de $B[1 : 0]$.

$B[1 : 0]$	Resultado
00	$a_3 a_2 a_1 a_0$
01	$0 a_3 a_2 a_1$
10	$0 0 a_3 a_2$
11	$0 0 0 a_3$

4. Implementación de las operaciones mediante compuertas

La suma de 4 bits es un ripple carry: cuatro full adders uno detrás del otro. El Cin del bit menos significativo vale 0. Cada full adder tiene dos xor (para S), dos and y un or (para Cout). El Cout de un bit entra como Cin del siguiente.

La resta usa el mismo sumador. Primero invert4 niega los cuatro bits de B con not. Ese valor entra al adder4 con Cin = 1, que es lo mismo que $A + \overline{B} + 1$. La resta inversa no agrega otro circuito: se cambian A y B de lugar y se aplica la misma idea.

El reset deja la salida en 0000. En la ALU ese camino se habilita cuando el código es 000.

Los shifts no corren los bits con un operador de Verilog. Cada bit de salida pasa por un multiplexor 4:1. El selector es $B[1:0]$ y elige el bit de A que corresponde o un 0 si ese lugar hay que rellenarlo. Van cuatro mux para el left y cuatro para el right.

Al final, las seis operaciones se calculan al mismo tiempo. Un decodificador arma un cable por cada código (000 a 101). Esos cables habilitan, bit a bit, la salida que corresponde. Lo que no se eligió queda en 0 y un or junta el resultado. Eso reemplaza un case.

El registro reg4 guarda esos 4 bits. Copia la salida de la ALU en el flanco de clk solo si ejecutar vale 1. Si ejecutar está en 0, el número que se ve no cambia.

5. GTKWave y Simulación

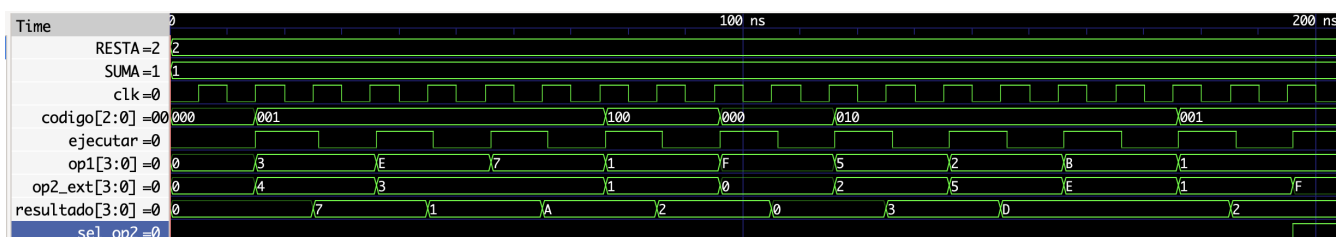


Figura 2: Simulación GTKWave

Fuente: Elaboración propia, simulación con Icarus Verilog y GTKWave.

La Figura 2 corresponde a una ejecución del testbench de la calculadora. Se observa el reloj, el código de operación, la señal ejecutar y la evolución de los operandos y del resultado. A lo largo de la traza se identifican sumas, un desplazamiento a la izquierda, el reinicio, restas y, al final, el uso del resultado anterior cuando sel_op2 vale 1.

El registro parte en un valor indefinido y se actualiza en cada pulso de ejecutar. La simulación se realizó con Icarus Verilog y la visualización de las señales con GTKWave, a partir del archivo de ondas generado por el testbench.

6. FPGA



Figura 3: Nandland Go Board
Fuente: Fotografía propia.

El diseño se programó en una Nandland Go Board (Lattice iCE40 HX1K). La interfaz sigue el enunciado: cuatro botones, LEDs para el código de operación y dos displays de siete segmentos para los valores.

El botón superior izquierdo incrementa el valor que se está ingresando. El inferior izquierdo lo disminuye. El superior derecho confirma el dato y avanza al siguiente paso. El inferior derecho activa `sel_op2`, de modo que el segundo operando sea el resultado anterior y no un valor nuevo.

El ingreso se hace en este orden. Primero se elige la operación; su código se muestra en los LEDs. Después se ingresan `op1` y `op2`. En los displays, el de la izquierda indica el signo (apagado si el número es positivo o cero, y un menos si es negativo) y el de la derecha muestra la magnitud en hexadecimal. Una vez confirmados ambos operandos, se ejecuta la operación y el resultado aparece en los mismos displays. Si se vuelve a presionar el botón superior derecho, el sistema regresa al estado inicial.

7. Conclusión

Se diseñó e implementó una calculadora de 4 bits capaz de operar con números positivos y negativos en complemento a dos. El circuito incluye las seis operaciones pedidas en la guía: reinicio, suma, resta, resta inversa, shift left y shift right. El segundo operando puede ser un valor externo o el resultado guardado, según `sel_op2`, y la salida queda almacenada en un registro de 4 bits que se actualiza con ejecutar.

La implementación se hizo a nivel de compuertas. El full adder es la base de la suma y, con la entrada invertida y $C_{in} = 1$, también de las dos restas. Los desplazamientos no usan operadores de Verilog: cada bit se arma con un multiplexor 4:1 controlado por $B[1 : 0]$. La ALU calcula los seis caminos en paralelo y un decodificador deja pasar solo el que corresponde al código.

Los números se interpretan siempre en 4 bits. Si una cuenta se sale del rango -8 a 7 , se conservan los cuatro bits menos significativos. Por eso $7 + 1$ queda en 1000 (-8) y $5 + 5$ en 1010 (-6). Eso no es un fallo del circuito; es la regla del enunciado.

La simulación con Icarus Verilog y GTKWave permite ver el reloj, los pulsos de ejecutar, el cambio de código y cómo se actualiza el resultado. En la traza se distinguen sumas, restas, reset, un shift left y el encadenamiento cuando `sel_op2` vale 1.

El mismo diseño se cargó en la Nandland Go Board. Los botones sirven para elegir operación y operandos, los LED muestran el código y los displays presentan el signo y la magnitud. Así se cierra el flujo del proyecto: de las tablas y los mapas de Karnaugh al Verilog, y de la simulación al hardware.